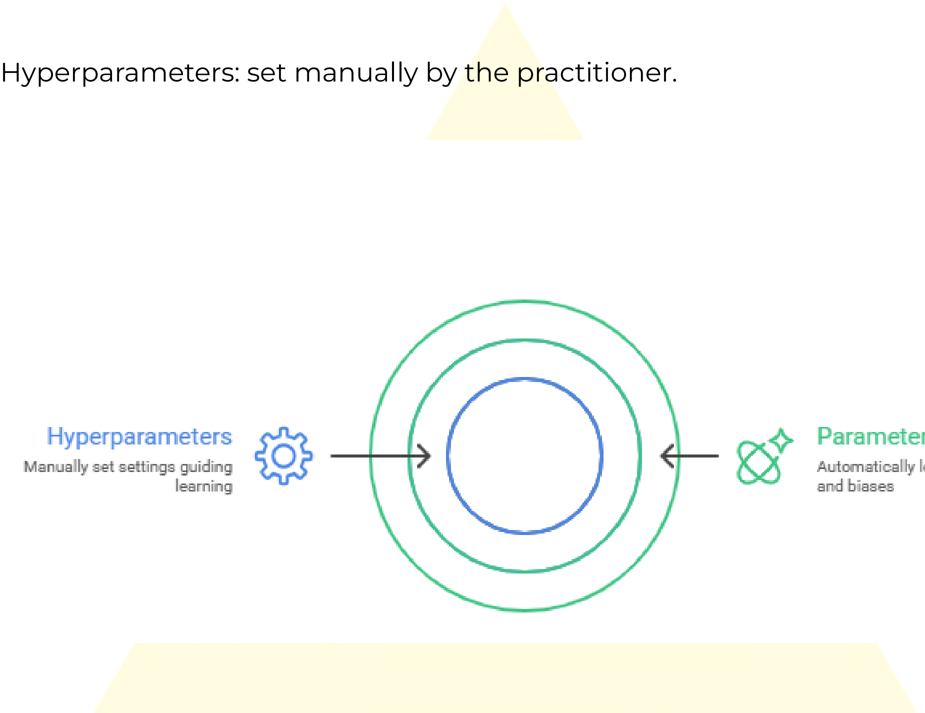
Hyperparameters and Training: Foundations of Deep Learning

Overview

In deep learning, hyperparameters are configuration settings chosen before training a neural network. They control how the learning process works and determine whether a model learns efficiently or poorly.

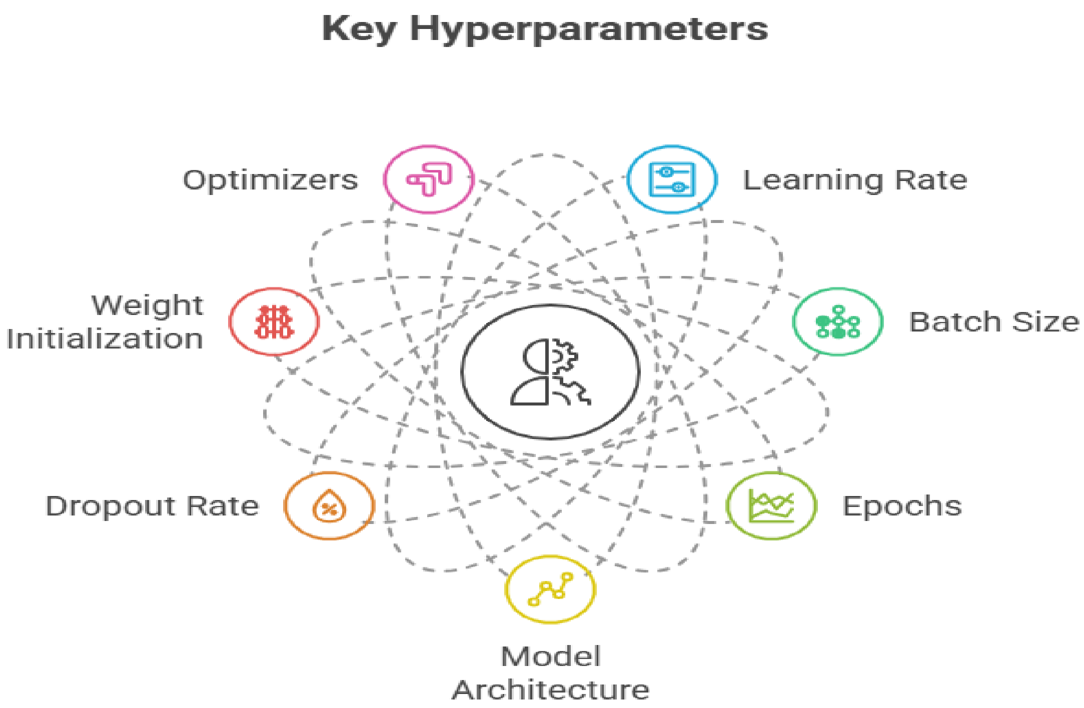
- Parameters (weights & biases): learned automatically during training.
- Hyperparameters: set manually by the practitioner.



Example: Training a student

- Parameters = knowledge gained from study.
- Hyperparameters = study plan (hours/day, practice tests, breaks).

Important Hyperparameters



2.1 Learning Rate (α)

- Controls how big a step the optimizer takes when updating weights.
- Too high → overshooting the minimum.
- Too low → very slow learning.

Analogy: Walking down a hill.

- Large steps = risk of missing the valley.
- Small steps = safer but slower.

2.2 Batch Size

- Number of samples processed before updating weights.
- Small batch (SGD) → noisy updates, better generalization.
- Large batch → smoother updates, faster but may overfit.
- Mini-batch (32–128) → commonly used, balance of speed & accuracy.

2.3 Epochs

- One epoch = one full pass through the dataset.
- Few epochs → underfitting.
- Too many epochs → overfitting.
- Solution: Use early stopping (stop when validation loss increases).

2.4 Model Architecture

- Number of layers and neurons per layer.
- Too small → underfits.
- Too large → overfits & slows training.
- Must balance complexity and generalization.

2.5 Dropout Rate

- Randomly “drops” neurons during training.
- Prevents over-reliance on specific neurons.
- Typical values = 0.2 to 0.5.

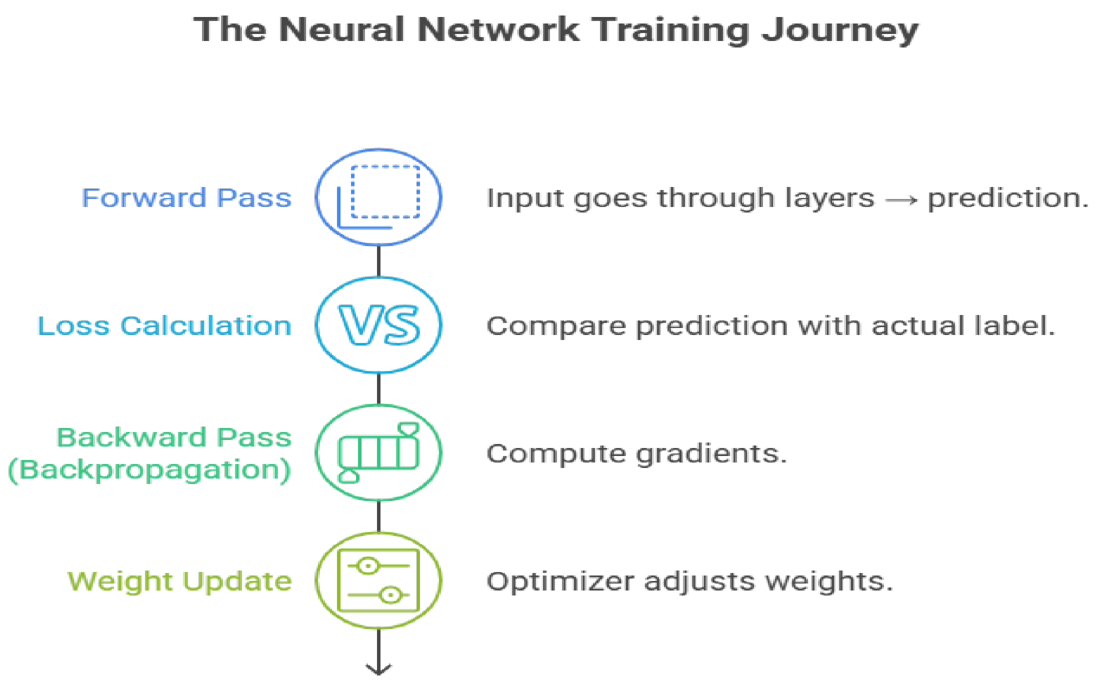
2.6 Weight Initialization

- Proper initialization avoids vanishing/exploding gradients.
- Common methods: Xavier, He initialization.

2.7 Optimizers

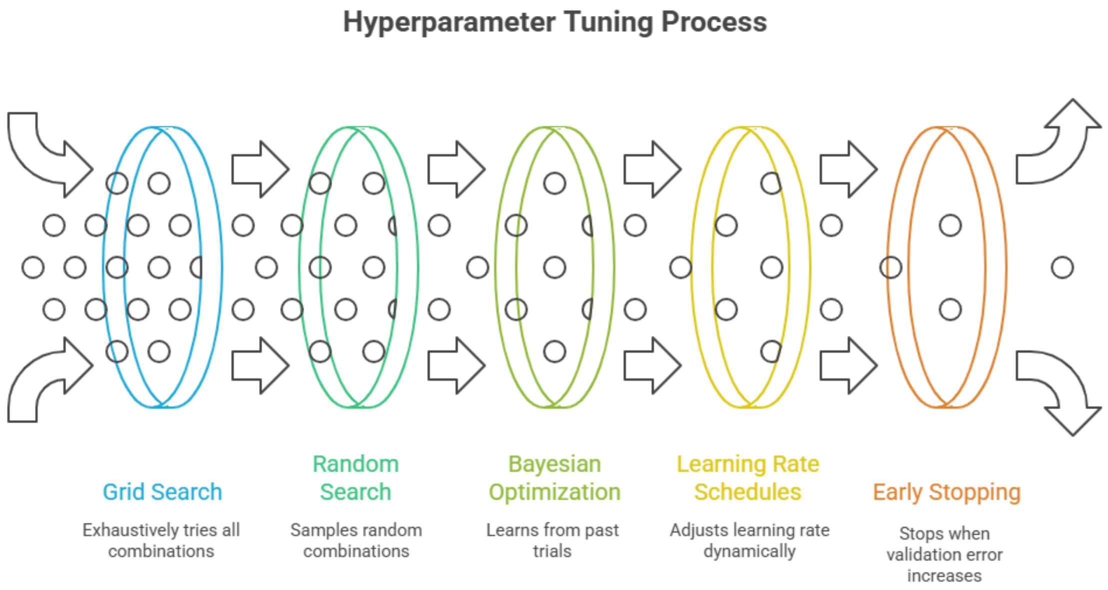
- Decide how weights are updated.
- SGD: simple, works well but slower.
- Adam: adaptive learning rate, very popular.
- RMSProp: useful for recurrent networks.

Training Process



1. Forward Pass → Input goes through layers → prediction.
2. Loss Calculation → Compare prediction with actual label.
3. Backward Pass (Backpropagation) → Compute gradients.
4. Weight Update → Optimizer adjusts weights.
5. Repeat for multiple epochs until the model converges.

Hyperparameter Tuning Strategies



- Grid Search: Try all possible combinations.
- Random Search: Sample random combinations.
- Bayesian Optimization: Smarter search, learns from past trials.
- Learning Rate Schedules: Reduce learning rate when progress slows.
- Early Stopping: Stop when validation error increases.

Real-World Example

Training a CNN for cats vs dogs classification:

- Learning rate = 0.001
- Batch size = 64
- Epochs = 25

- Optimizer = Adam
- Dropout = 0.5

Good tuning here could boost accuracy from 70% → 90%.

Key Takeaways

- Hyperparameters = rules that guide training.
- Most critical: learning rate, batch size, epochs, architecture, dropout.
- Training = forward pass → loss → backprop → optimizer update.
- Smart tuning = better accuracy, faster training, less overfitting.